

OPTAM-MPC

Hands-on Application Tutorial

Part 3: Rebuild the Heat Exchanger as NMPC

Nonlinear equations, model contract, domain checks, independent calculations and honest evidence

Applies to	OPTAM-MPC Nonlinear Starter v0.2.0
Prerequisite	Tutorial Parts 1 and 2, or equivalent familiarity
Issue	Draft 1 — 6 September 2026
Method	Predict → configure → calculate → validate → run → record evidence

The frozen starter and user manual remain authoritative.

Why Part 3 keeps the same heat exchanger

Part 2 taught multivariable interaction with seven linear paths. Part 3 keeps the same measured signals, valves and control objectives, but replaces the path bank with one nonlinear state-transition function. The learner can therefore see exactly what the nonlinear model changes without simultaneously learning a new process.

Part 2: linear	Part 3: nonlinear
Seven independent first-order paths	One coupled transition function
Constant local gains	Gain varies with operating point
Path-specific lags and delays	Two lumped output lags; no explicit delay
TOML contains the model	TOML parameters plus engineer-owned model.py
Eight path checks	Five whole-model next-sample checks
Linear superposition applies	Responses need not add

HONEST SCOPE This example is deliberately only mildly nonlinear. It teaches the workflow; it is not evidence that NMPC will outperform linear MPC on every plant.

Learning outcomes

- Explain the four nonlinear mechanisms in engineering terms.
- Write a small CasADi-compatible transition function without changing the controller core.
- Define safe model domains and parameter checks.
- Calculate at least one next sample independently before trusting Python.
- Distinguish model consistency, simulation acceptance and plant validity.
- Explain why velocity-form feedback remains valuable with model mismatch.

Module 9 — See what is genuinely nonlinear

Same plant — a different prediction model

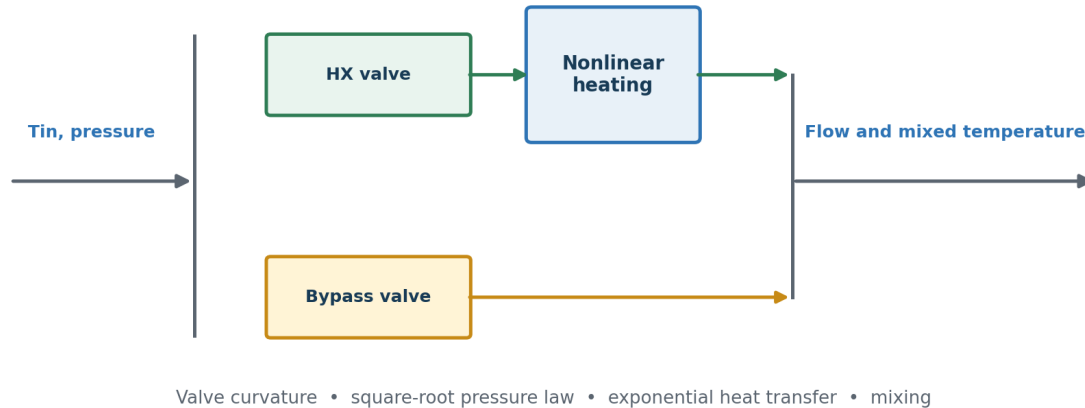


Figure 1. The physical arrangement is unchanged; the steady calculation is now coupled and nonlinear.

9.1 Variables and operating range

Role	Definition and tutorial range
CV flow	60 m ³ /h nominal; protected 45–75; model domain 30–90
CV temperature	64 °C nominal; protected 55–75; domain 45–85
HX and bypass MVs	50% nominal; hard 35–65%; move ≤1 point/sample
Inlet-temperature DV	40 °C nominal; domain 37–43
Pressure DV	5 barg nominal; domain 4.6–5.4

9.2 The four nonlinear mechanisms

1. Valve curvature: each branch flow includes $c(v-1)^2$.
2. Pressure: both branch flows scale with the square root of differential pressure.
3. Heat transfer: heated temperature contains $\exp(-A/qh)$, so effectiveness changes with flow.
4. Mixing: outlet temperature is a flow-weighted ratio, not an independent additive path.

PAUSE AND PREDICT At higher HX-branch flow, does the branch have more or less time to approach the hot-side temperature? What happens to heat-exchanger effectiveness?

9.3 The equations

```

h = hx_valve / 50    b = bypass_valve / 50
s = sqrt((pressure - 3) / 2)
qh = 30 * (h + c*(h - 1)^2) * s
qb = 30 * (b + c*(b - 1)^2) * s
Qeq = qh + qb
Theated = 100 - (100 - Tin) * exp(-A/qh)
Teq = (qh*Theated + qb*Tin) / Qeq

```

At the nominal point $c=0.1$ and $A=30 \ln(5)$. Both branches carry $30 \text{ m}^3/\text{h}$, the heated branch is 88°C , and mixing equal branch flows gives 64°C .

Module 10 — Understand the nonlinear model contract

What `transition()` must calculate for one sample

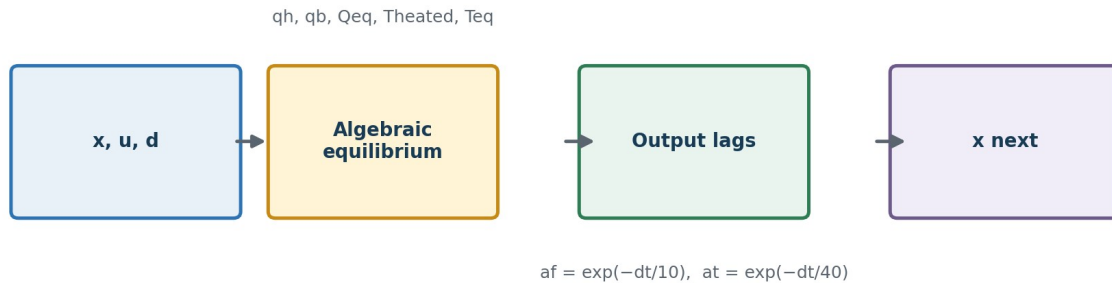


Figure 2. `transition()` returns the absolute next state after one controller sample.

10.1 The engineer owns only one small Python extension

Copy `model_template.py` beside `application.toml` and rename it `model.py`. The starter still owns optimisation, velocity-form prediction, simulation, OPC, plots and reports. The engineer supplies signal order, parameter validation and the physical transition equations.

```
STATE_IDS = ("flow", "temperature")
MV_IDS = ("hx_valve", "bypass_valve")
DV_IDS = ("inlet_temperature", "upstream_pressure")
```

ORDER IS A CONTRACT The ID tuples must match the TOML order exactly. A physically correct formula connected to the wrong ID is still a wrong controller model.

10.2 Numeric and symbolic compatibility

During validation the same function is evaluated with ordinary numbers and CasADi symbolic expressions. Use `casadi.exp` and `casadi.sqrt` for expressions involving `x`, `u` or `d`. Do not hide equations in unreviewed helper modules.

```
def transition(x, u, d, p, dt):
    pressure_factor = ca.sqrt((d["upstream_pressure"] - p["downstream_pressure"])
                              / (p["pressure_nominal"] - p["downstream_pressure"]))
    # calculate qh, qb, total, heated and mixed
    af = ca.exp(-dt / p["flow_tau_seconds"])
    at = ca.exp(-dt / p["temperature_tau_seconds"])
    return {"flow": af*x["flow"] + (1-af)*total,
            "temperature": at*x["temperature"] + (1-at)*mixed}
```

10.3 Absolute state—not an increment

`transition()` must return the predicted absolute CV values one sample later. The controller itself applies velocity-form anchoring. Returning only Δflow and $\Delta\text{temperature}$ would double-confuse the state update and is a fundamental implementation error.

Module 11 — Build from the generic templates

11.1 Create an independent workspace

```
mkdir applications\my_nonlinear_hx
copy application_template.toml applications\my_nonlinear_hx\application.toml
copy model_template.py applications\my_nonlinear_hx\model.py
copy simulation_template.toml applications\my_nonlinear_hx\simulation.toml
copy opc_template.toml applications\my_nonlinear_hx\opc.toml
```

DO NOT EDIT THE FROZEN RUNTIME Only the copied application files are learner-owned. Keep the starter, generic templates and completed reference unchanged.

11.2 Configure the controller

```
[controller]
prediction_form = "velocity"
prediction_horizon_samples = 30
control_horizon_samples = 10
maximum_solver_cpu_seconds = 4
use_measured_disturbances = true
```

Setting	Part 3 value
Sample time	5 seconds
CV normalisations	5 m ³ /h; 2 °C
CV weights	1; 1
MV bounds	35–65%
MV move limits	±1 point/sample
MV move weights	0.5; 0.5
Protected-limit weights	1000; 1000

11.3 Declare parameters and domains

```
[model]
file = "model.py"
[model.parameters]
branch_flow_nominal = 30
valve_nominal = 50
pressure_nominal = 5
downstream_pressure = 3
inlet_temperature_nominal = 40
hot_temperature = 100
heat_transfer_flow = 48.28313737302301
valve_curvature = 0.1
flow_tau_seconds = 10
temperature_tau_seconds = 40

[model.domain]
flow = [30, 90]
temperature = [45, 85]
hx_valve = [35, 65]
bypass_valve = [35, 65]
inlet_temperature = [37, 43]
upstream_pressure = [4.6, 5.4]
```

DOMAIN MEANS MATHEMATICAL VALIDITY The lower pressure bound must remain above downstream pressure so the square root is real. A domain is not a substitute for operating limits or process safety protection.

Module 12 — Verify the model before tuning

12.1 Independent cold-inlet calculation

With both valves at 50%, pressure 5 barg and T_{in} changed from 40 to 37 °C, both branch flows remain 30 m³/h. The heated branch becomes $100 - 63/5 = 87.4$ °C and the mixed equilibrium becomes 62.2 °C.

```
temperature_next
= 64 + (1 - exp(-5/40)) * (62.2 - 64)
= 63.78849442465227 degC

flow_next = 60.0 m3/h
```

WHY THIS MATTERS The expected result is derived independently of model.py. Copying the program's own output into the check would only prove that the program agrees with itself.

12.2 Add whole-model checks

- Nominal point remains exactly stationary.
- HX-valve step changes both outputs in the expected direction.
- Bypass-valve step raises flow and lowers temperature.
- Cold-inlet case matches the independent calculation above.
- High-pressure case verifies square-root pressure dependence and coupled temperature response.

12.3 Validate

```
validate_application.bat applications\my_nonlinear_hx\application.toml
applications\my_nonlinear_hx\simulation.toml applications\my_nonlinear_hx\opc.toml
```

Fresh validation evidence	Result
Numeric/symbolic domain probes	77
Independent nonlinear checks	5 of 5 PASS
Nominal solver smoke test	PASS
Configuration/model consistency	PASS

DELIBERATE FAILURE Reverse the sign of the bypass contribution in the mixing equation. Predict which checks fail, validate, record the diagnostic, then restore the equation.

Module 13 — Run and interpret the nonlinear controller

13.1 Combined scenario

Sample	Change
20 / 100	Flow target 60→64→60 m ³ /h
180 / 260	Temperature target 64→66→64 °C
340 / 420	Tin 40→37→40 °C
500 / 580	Pressure 5→5.4→5 barg

```
run_simulation.bat applications\my_nonlinear_hx\application.toml
applications\my_nonlinear_hx\simulation.toml --output results\combined
```

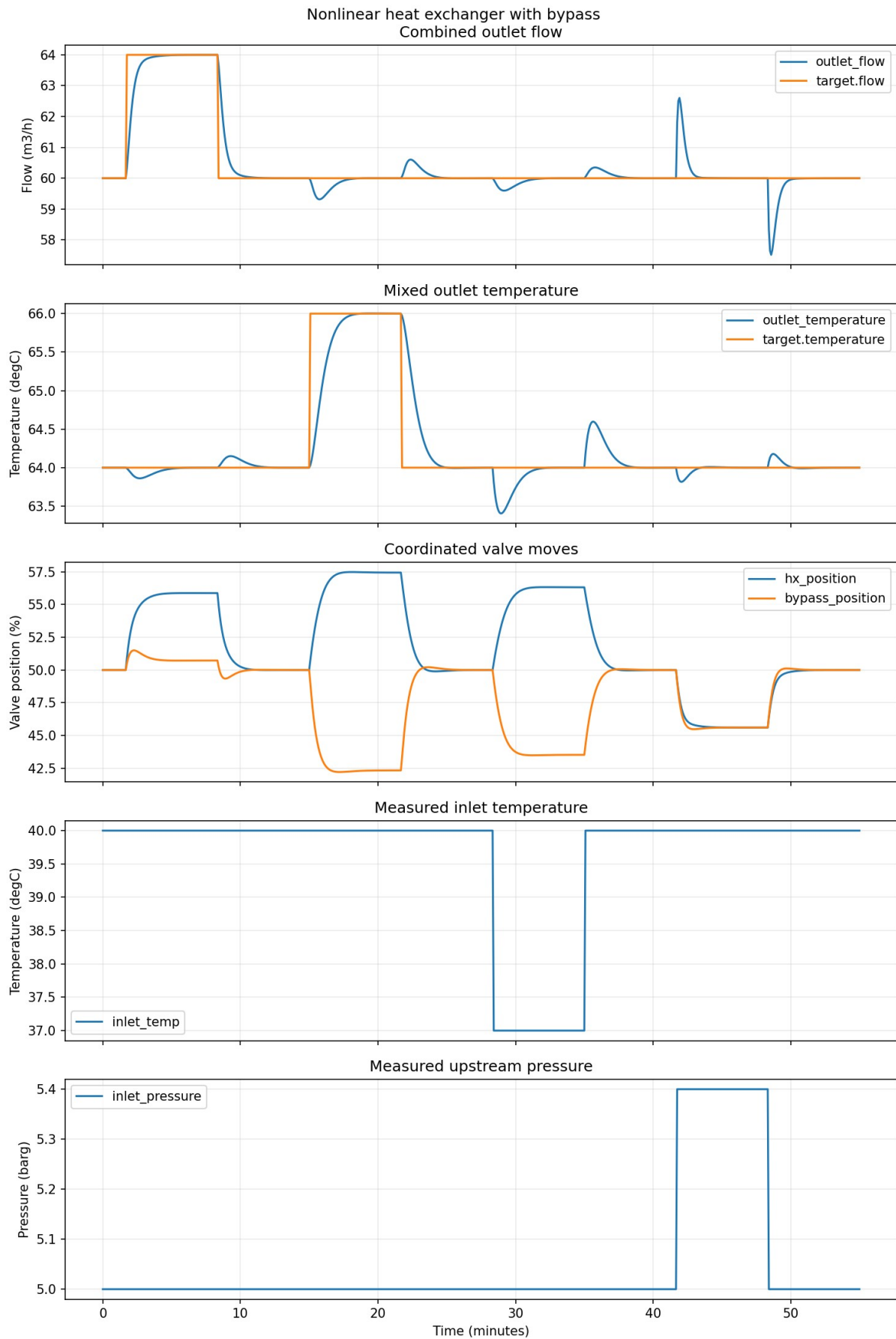


Figure 3. Verified nonlinear combined scenario: setpoint and measured-disturbance responses.

Archived reference evidence	Value
Flow range	57.5168–64.0001 m ³ /h
Temperature range	63.4056–66.0027 °C
Valve ranges	HX 45.60–57.48%; bypass 42.21–51.50%
Maximum moves	HX 0.9316; bypass 1.0000 points/sample
Acceptance	PASS

13.2 Read the valve coordination

A flow target change mainly drives common valve movement. A temperature target change mainly drives differential movement. Pressure affects both branch flows together but also changes exchanger effectiveness; the nonlinear model calculates these effects jointly rather than adding fixed paths.

DO NOT COMPARE FIGURES 2 AND 3 AS A CONTEST Part 2 and Part 3 use different lags, delays, bounds, temperature excursion and tuning. This tutorial comparison is structural, not a fair performance benchmark.

Module 14 — Feedforward, mismatch and limits of evidence

14.1 Feedforward on/off

Repeat the controlled comparison from Part 2. Only use `_measured_disturbances` changes; plant events and acceptance checks remain identical.

```
run_simulation.bat applications\my_nonlinear_hx\application.toml
applications\my_nonlinear_hx\disturbances.toml --output results\ff_on
run_simulation.bat applications\my_nonlinear_hx\application_no_feedforward.toml
applications\my_nonlinear_hx\disturbances.toml --output results\ff_off
```

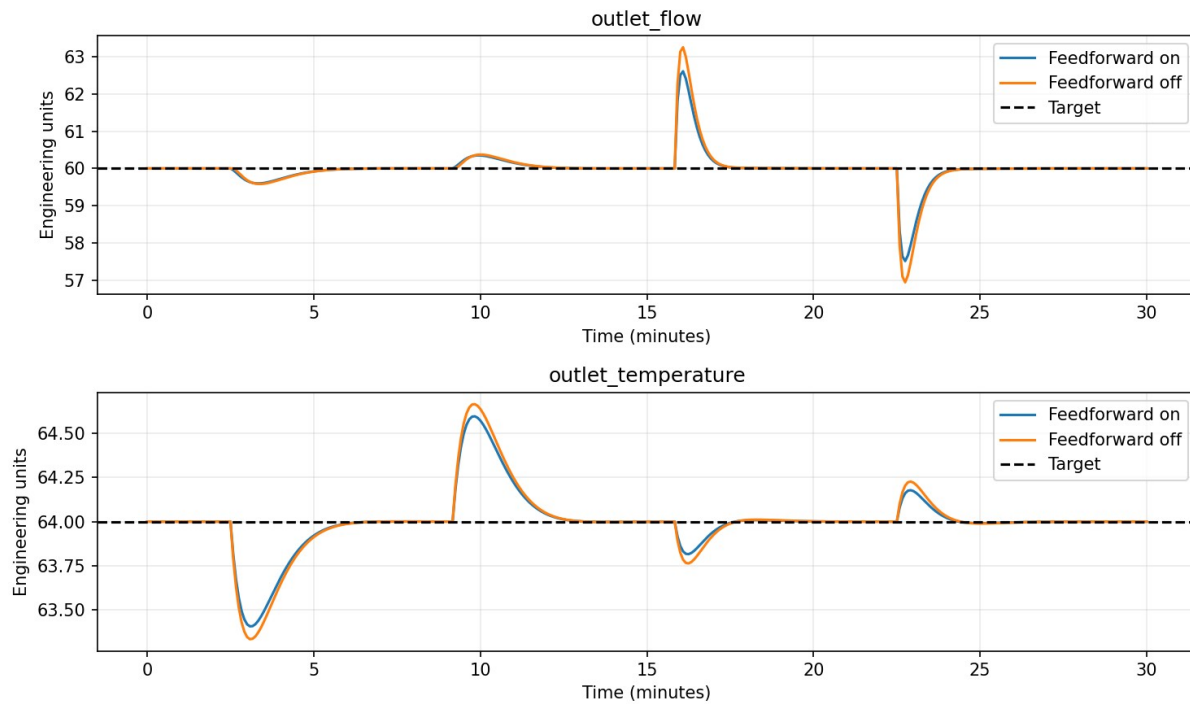


Figure 4. Measured-DV feedforward reduces the disturbance error in this verified example.

Final five-minute max error	Feedforward on / off
Flow	0.00785 / 0.00906 m ³ /h
Temperature	0.00910 / 0.01038 °C
Acceptance	PASS / PASS

14.2 Modest mismatch

The plant time constants increase by 20% and valve curvature doubles from 0.1 to 0.2. The controller remains nominal. This is a separate plant instance using the same equation form, not independent validation against reality.

```
run_simulation.bat applications\my_nonlinear_hx\application.toml  
applications\my_nonlinear_hx\mismatch.toml --output results\mismatch
```

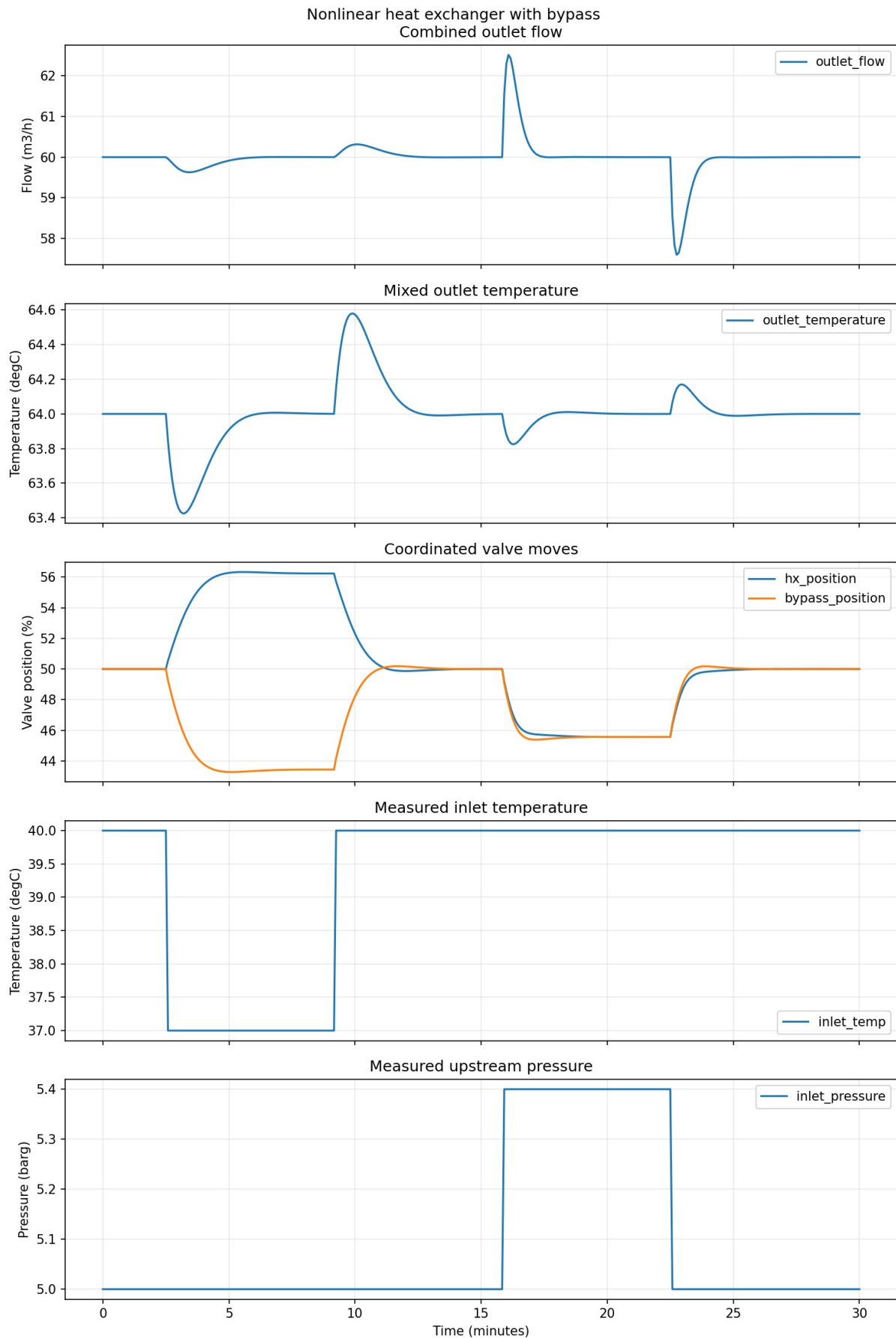


Figure 5. The modest mismatch case passes the declared final-window checks.

Mismatch evidence	Value
Final-window flow error	$\leq 0.00543 \text{ m}^3/\text{h}$
Final-window temperature error	$\leq 0.01111 \text{ }^\circ\text{C}$
Acceptance	PASS

FRANK CONCLUSION Passing this modest case is useful regression evidence. It is not a robustness certificate, plant validation, proof of global convergence or proof that all predicted states stay in a physically safe region.

Module 15 — OPC and completion

15.1 Run the accepted case through local OPC UA

```
run_digital_twin.bat applications\my_nonlinear_hx\application.toml
applications\my_nonlinear_hx\simulation.toml applications\my_nonlinear_hx\opc.toml
```

After Twin READY, open a second Command Prompt:

```
run_controller.bat applications\my_nonlinear_hx\application.toml applications\my_nonlinear_hx\opc.toml
```

Optional third window:

```
opc_tools.bat applications\my_nonlinear_hx\opc.toml trend applications\my_nonlinear_hx\simulation.toml
```

15.2 Completion checklist

- I created my own four application files from generic templates.
- I can explain every term in the nonlinear equations.
- I calculated one next sample independently.
- I understand absolute transition versus velocity-form anchoring.
- I validated numeric and symbolic evaluation and all declared domains.
- I ran the combined, feedforward and mismatch cases and recorded evidence.
- I can state why this is not a fair linear-versus-nonlinear benchmark.
- I did not edit the frozen controller or runtime.

15.3 Evidence for independent review

Record	What it tests
Time to first valid model.py	Usability for an APC engineer with limited Python
Failed check and diagnosis	Quality of validation feedback
Independent calculation	Protection against circular verification
Predicted valve directions	Physical understanding
Metrics and solver failures	Operational evidence
Suggested wording changes	Input to Draft 2

Where the tutorial series goes next

Part 3 completes the controlled transition from a 1×1 linear example to a 2×2 linear example and then to modest NMPC. A later tutorial can introduce the surge-vessel, heat-exchanger and nonlinear reactor system, where Arrhenius kinetics and residence-time trade-offs create a stronger reason for nonlinear economic optimisation.

RECOMMENDED SEQUENCE Test Parts 1–3 with an independent APC engineer before expanding the process. Their difficulties should determine what is clarified in Draft 2.

Appendix A — Supplied exercise files

- 00_blank_templates: untouched nonlinear application, model, simulation and OPC templates.
- 90_completed_reference: completed nonlinear exchanger files, design basis and verified outputs.
- fresh_verification: current validation, unit-test and simulation evidence produced for this tutorial.
- engineering_record.md: a structured place for predictions, calculations, failures, metrics and conclusions.

Appendix B — Minimum command set

```
validate_application.bat <application> [simulation] [opc]
run_simulation.bat <application> <simulation> [--output <folder>]
run_digital_twin.bat <application> <simulation> <opc>
run_controller.bat <application> <opc>
opc_tools.bat <opc> trend <simulation>
opc_tools.bat <opc> stop
```

Appendix C — What validation does not prove

- That the equations match a real plant.
- That unmeasured states are observable; this starter uses measured primary CV states.
- That all possible predicted steps remain inside the physical domain.
- That solver success guarantees a safe move.
- That passing one nominal and one modest mismatch case establishes production readiness.